R PROGRAMMING CHALLENGES

Topic 9: Return Values and Explicit return()

Name: Sameer Ahmed G

Register Number: 192421154

Department: B Tech Information Technology

Q136

Task: Compare square() using return() with a version relying on the last expression.

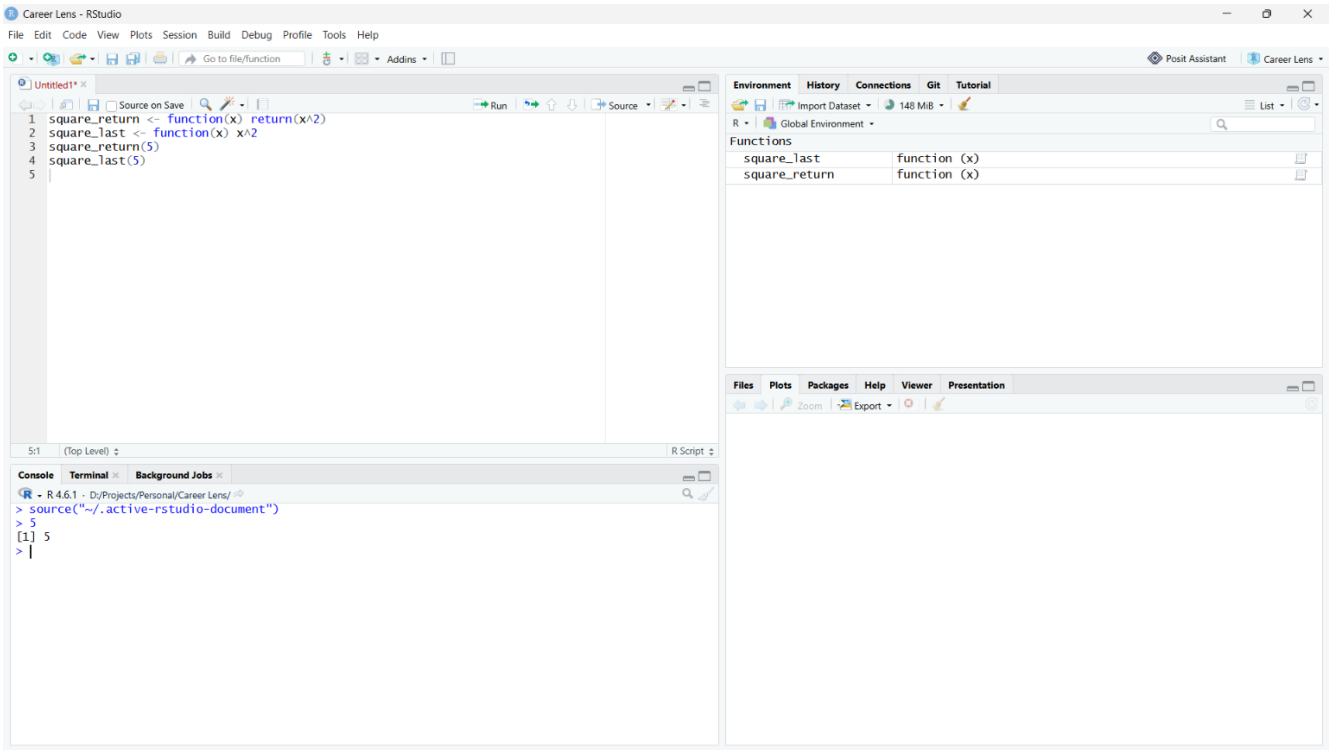
R Code:

```
square_return <- function(x) return(x^2)
square_last <- function(x) x^2
square_return(5)
square_last(5)
```

Expected Output:

```
[1] 25\n[1] 25
```

Result: Both functions return 25.



Q137

Task: Return NA with a message when b is zero.

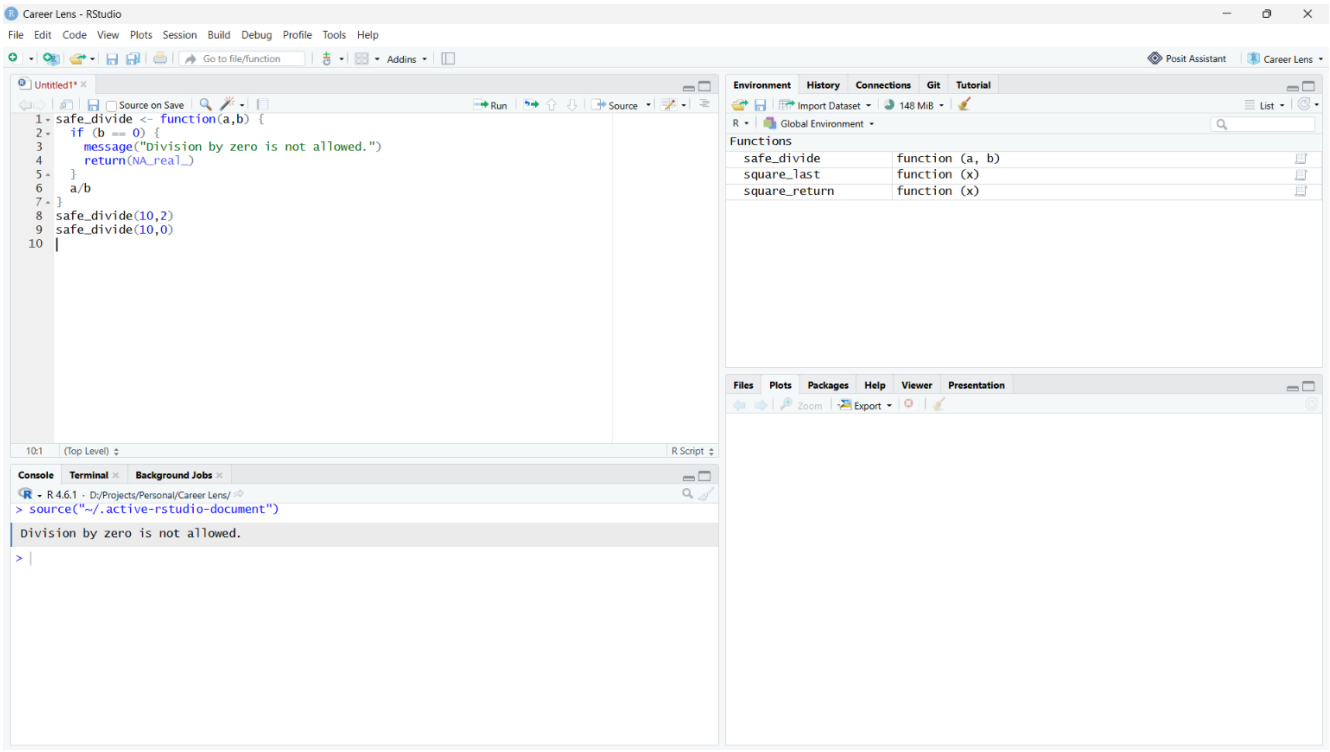
R Code:

```
safe_divide <- function(a,b) {
  if (b == 0) {
    message("Division by zero is not allowed.")
    return(NA_real_)
  }
  a/b
}
safe_divide(10,2)
safe_divide(10,0)
```

Expected Output:

```
[1] 5\nDivision by zero is not allowed.\n[1] NA
```

Result: The early return stops division when b equals zero.



Q138

Task: Print a log message and invisibly return TRUE; capture and print the value.

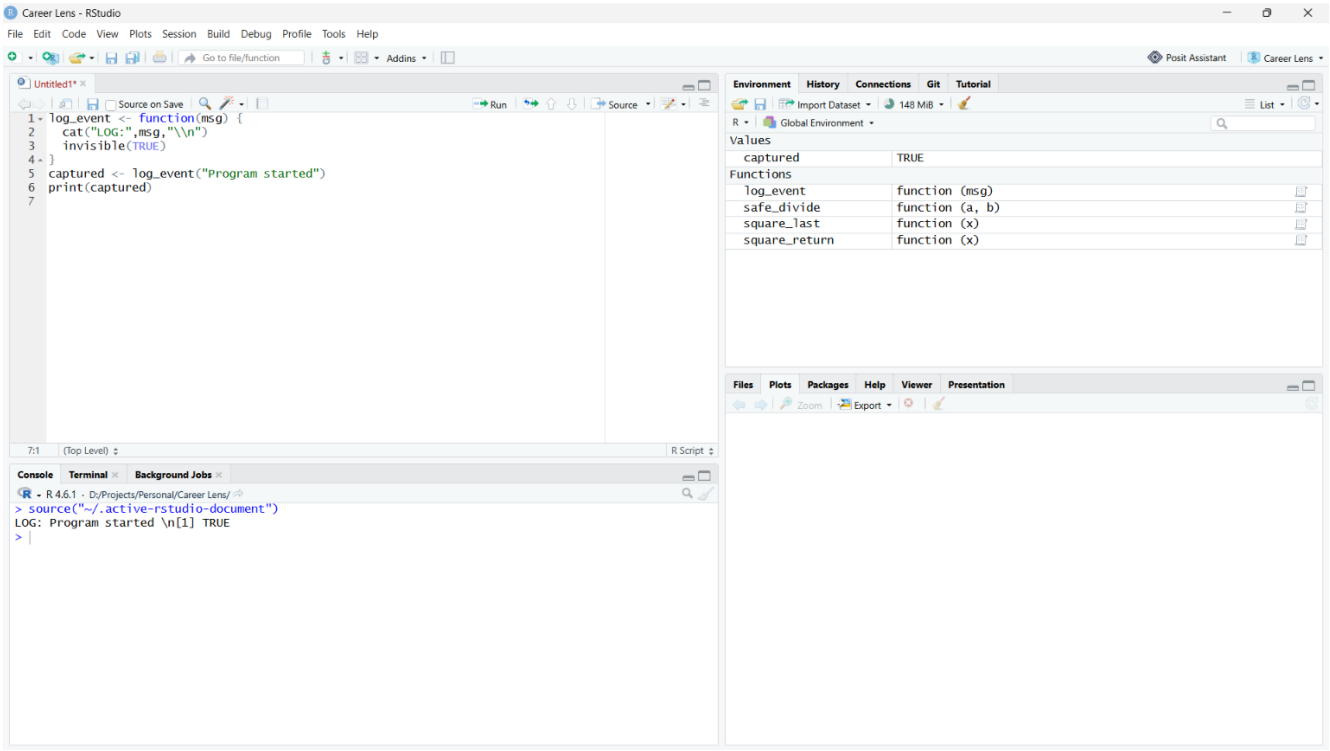
R Code:

```
log_event <- function(msg) {
  cat("LOG:",msg,"\\n")
  invisible(TRUE)
}
captured <- log_event("Program started")
print(captured)
```

Expected Output:

```
LOG: Program started\n[1] TRUE
```

Result: invisible() suppresses automatic display, but the returned value can still be captured.



Q139

Task: Use an if/else-if/else chain as the final expression, without return().

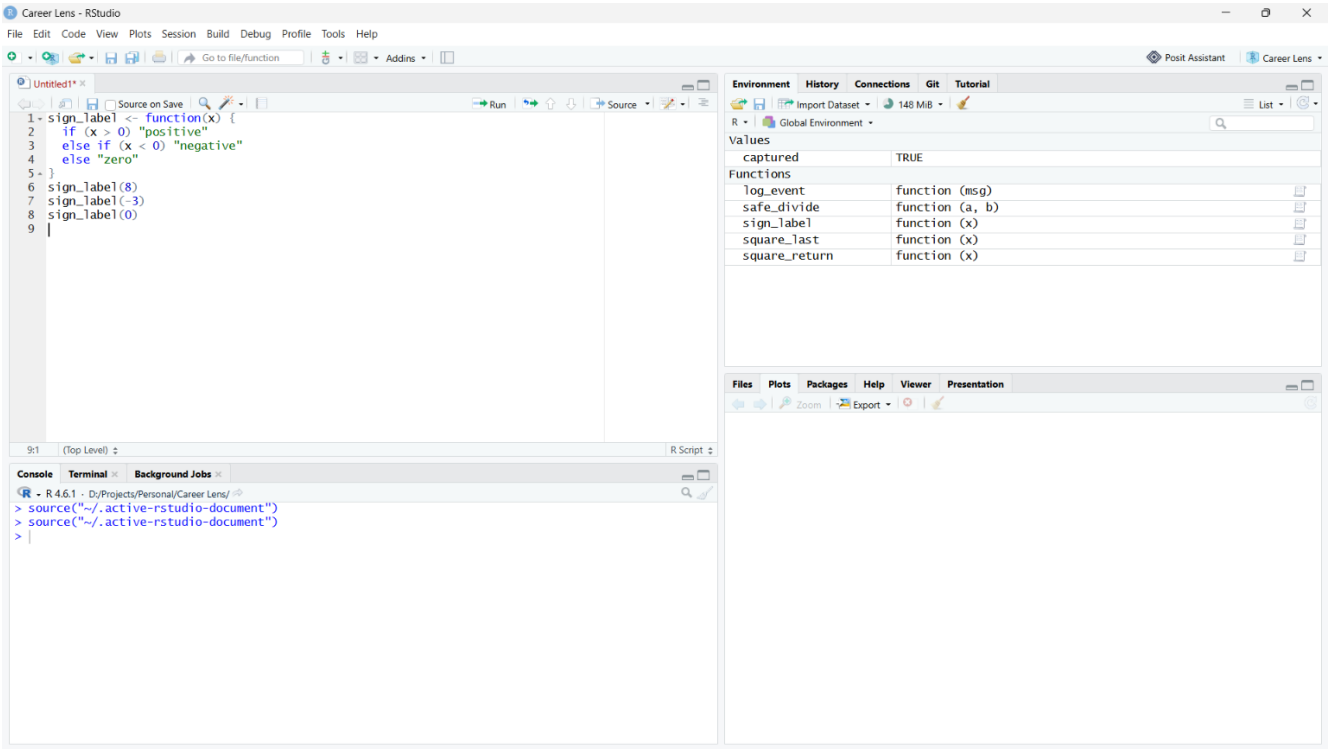
R Code:

```
sign_label <- function(x) {
  if (x > 0) "positive"
  else if (x < 0) "negative"
  else "zero"
}
sign_label(8)
sign_label(-3)
sign_label(0)
```

Expected Output:

```
[1] "positive"\n[1] "negative"\n[1] "zero"
```

Result: The final conditional expression is returned.



Q140

Task: Show that an assignment returns invisibly, then explicitly print the value.

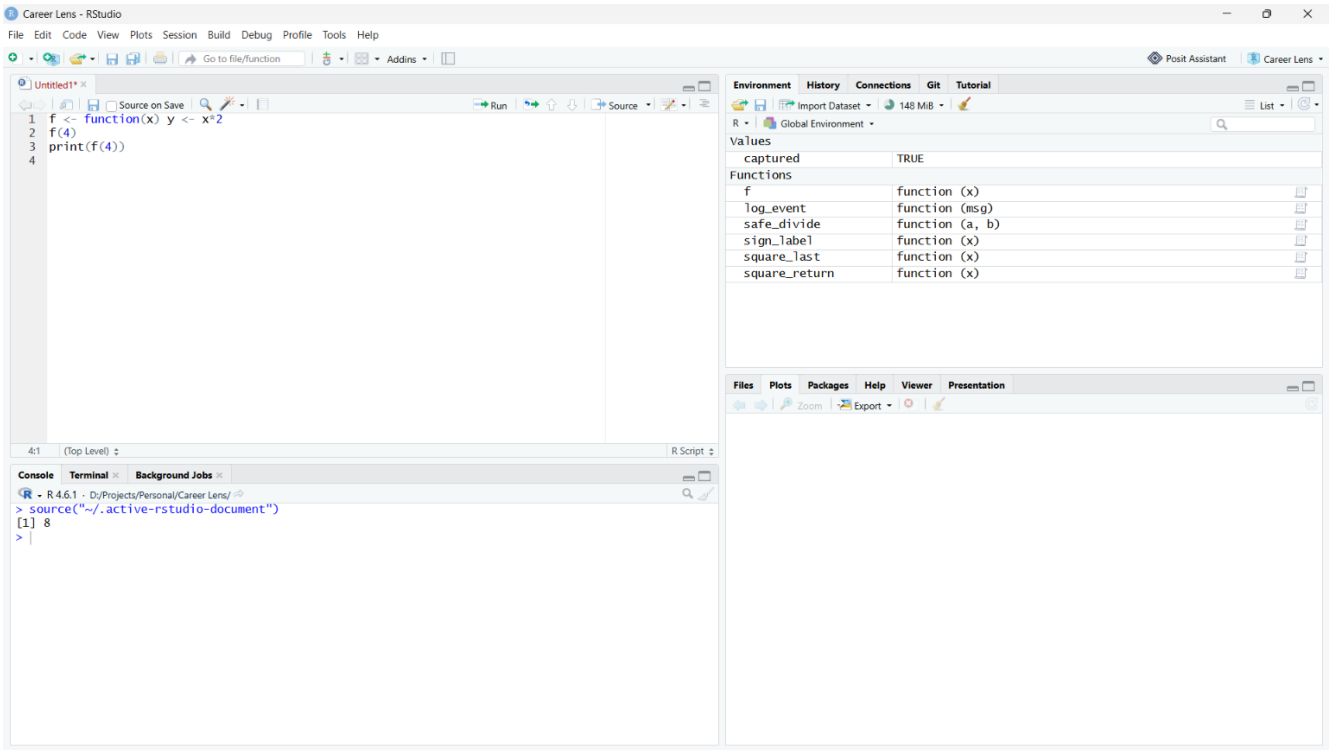
R Code:

```
f <- function(x) y <- x*2
f(4)
print(f(4))
```

Expected Output:

```
[1] 8
```

Result: The assignment returns its value invisibly; print() displays it.



Q141

Task: Return minimum and maximum as a named vector.

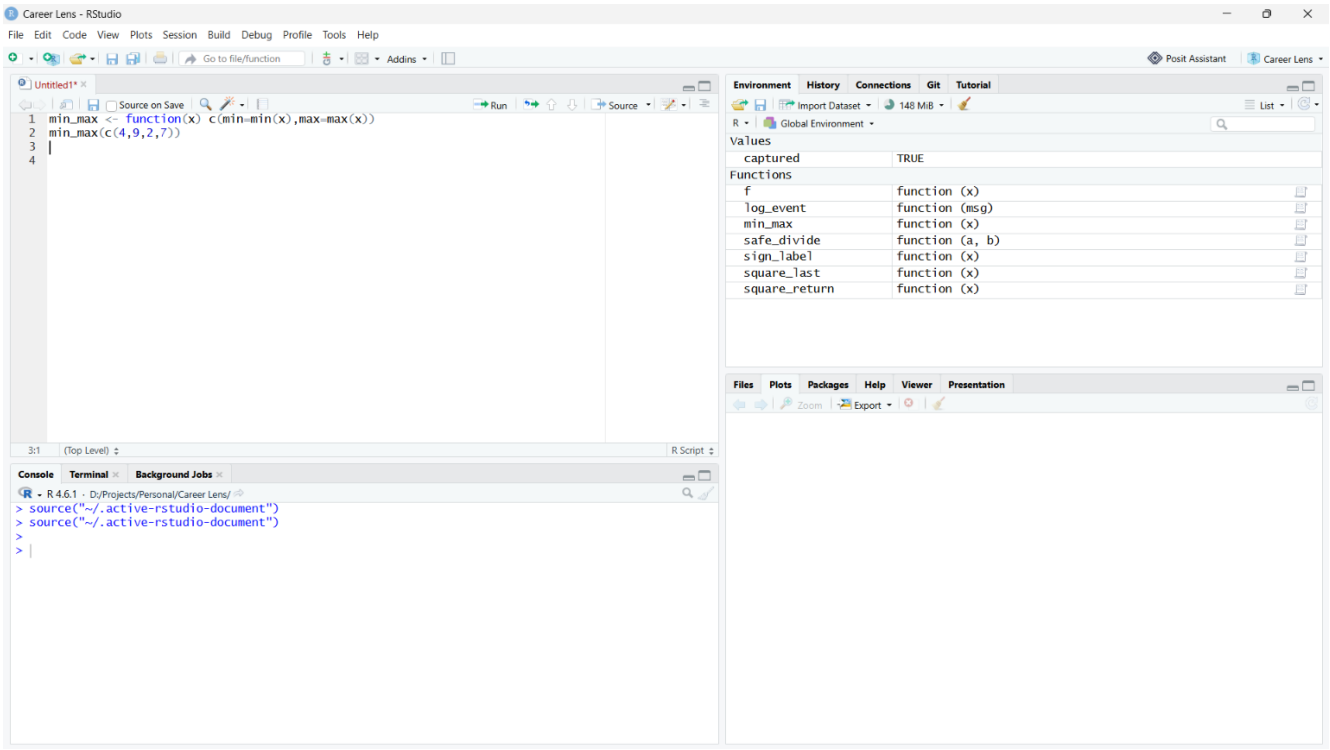
R Code:

```
min_max <- function(x) c(min=min(x),max=max(x))
min_max(c(4,9,2,7))
```

Expected Output:

```
min max\n  2   9
```

Result: The result contains named minimum and maximum values.



Q142

Task: Return the first negative value, or NULL when none exists.

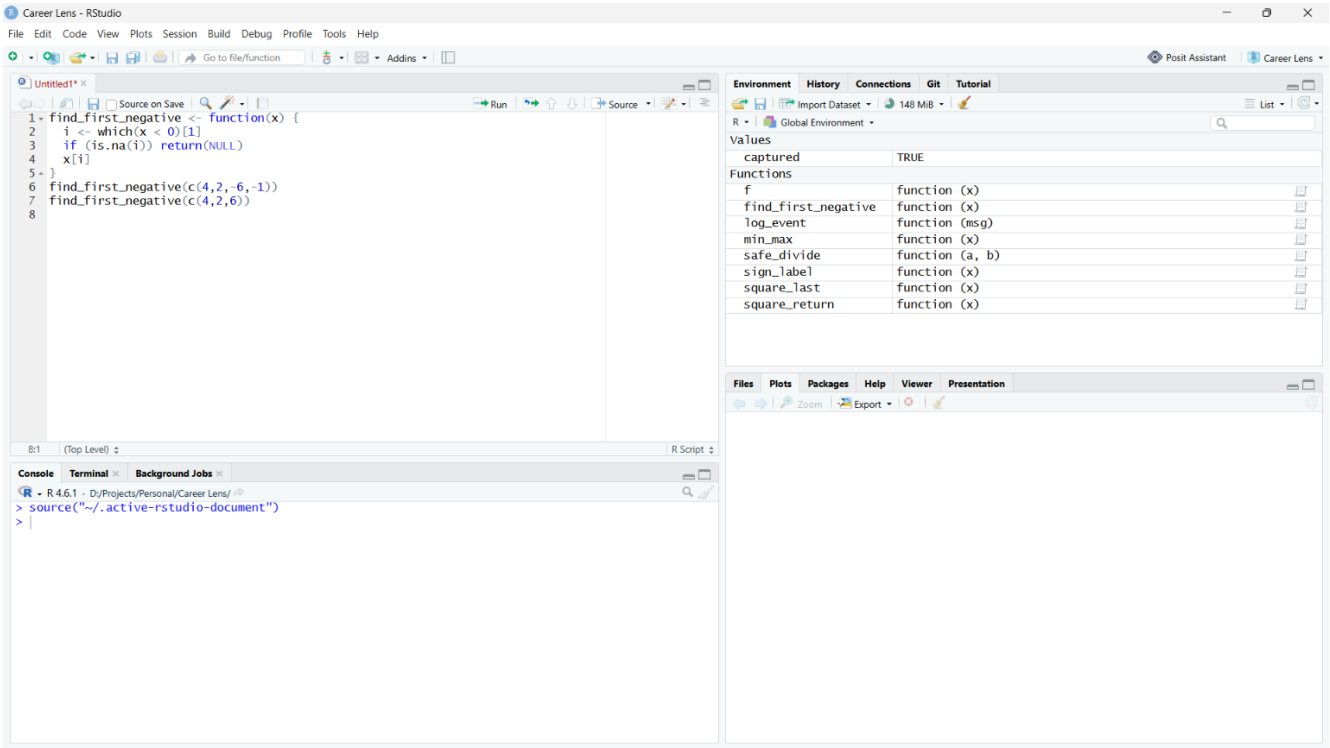
R Code:

```
find_first_negative <- function(x) {
  i <- which(x < 0)[1]
  if (is.na(i)) return(NULL)
  x[i]
}
find_first_negative(c(4,2,-6,-1))
find_first_negative(c(4,2,6))
```

Expected Output:

```
[1] -6\nNULL
```

Result: NULL indicates that no negative value was found.



Q143

Task: Return from a while loop as soon as a prime above n is found.

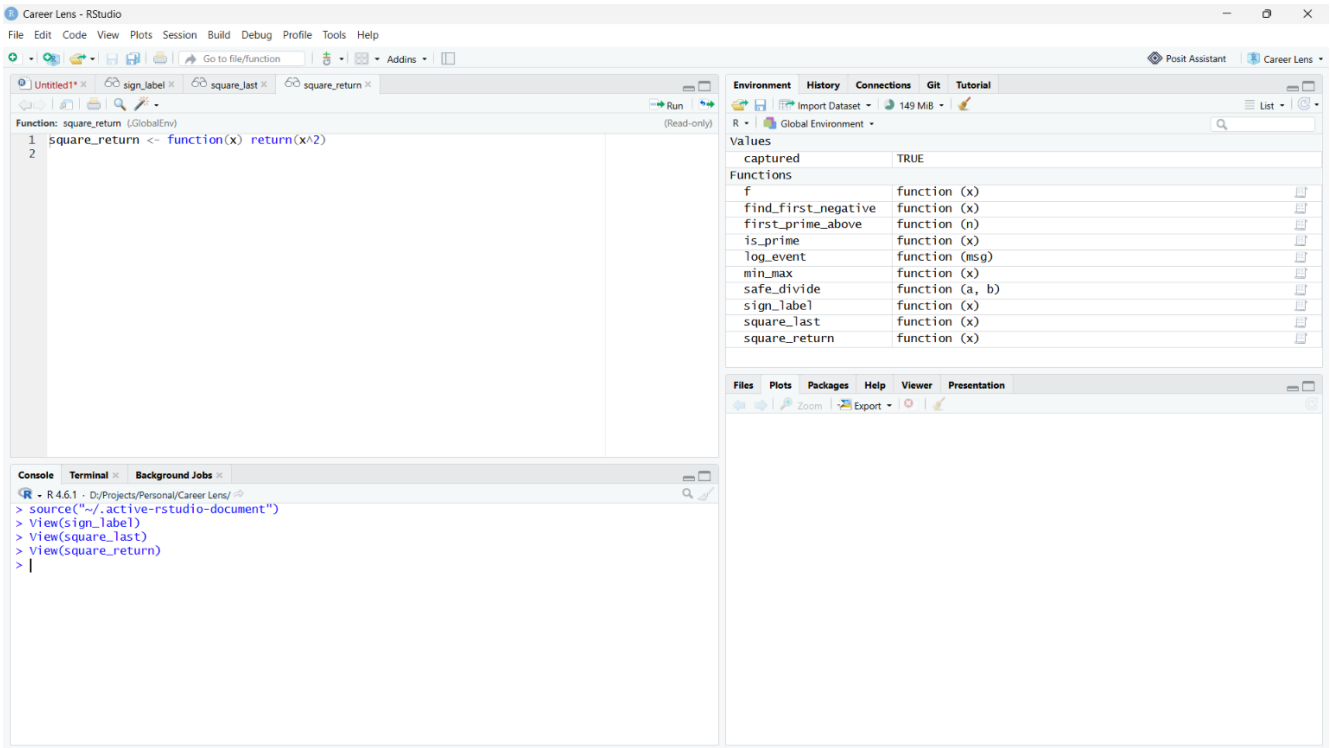
R Code:

```
is_prime <- function(x) {
  if (x < 2) return(FALSE)
  if (x == 2) return(TRUE)
  for (i in 2:floor(sqrt(x))) {
    if (i > sqrt(x)) break
    if (x %% i == 0) return(FALSE)
  }
  TRUE
}
first_prime_above <- function(n) {
  candidate <- n+1
  while (TRUE) {
    if (is_prime(candidate)) return(candidate)
    candidate <- candidate+1
  }
}
first_prime_above(10)
```

Expected Output:

```
[1] 11
```

Result: return() exits the function as soon as a prime is found.



Q144

Task: Compare printing a value with returning it for further calculation.

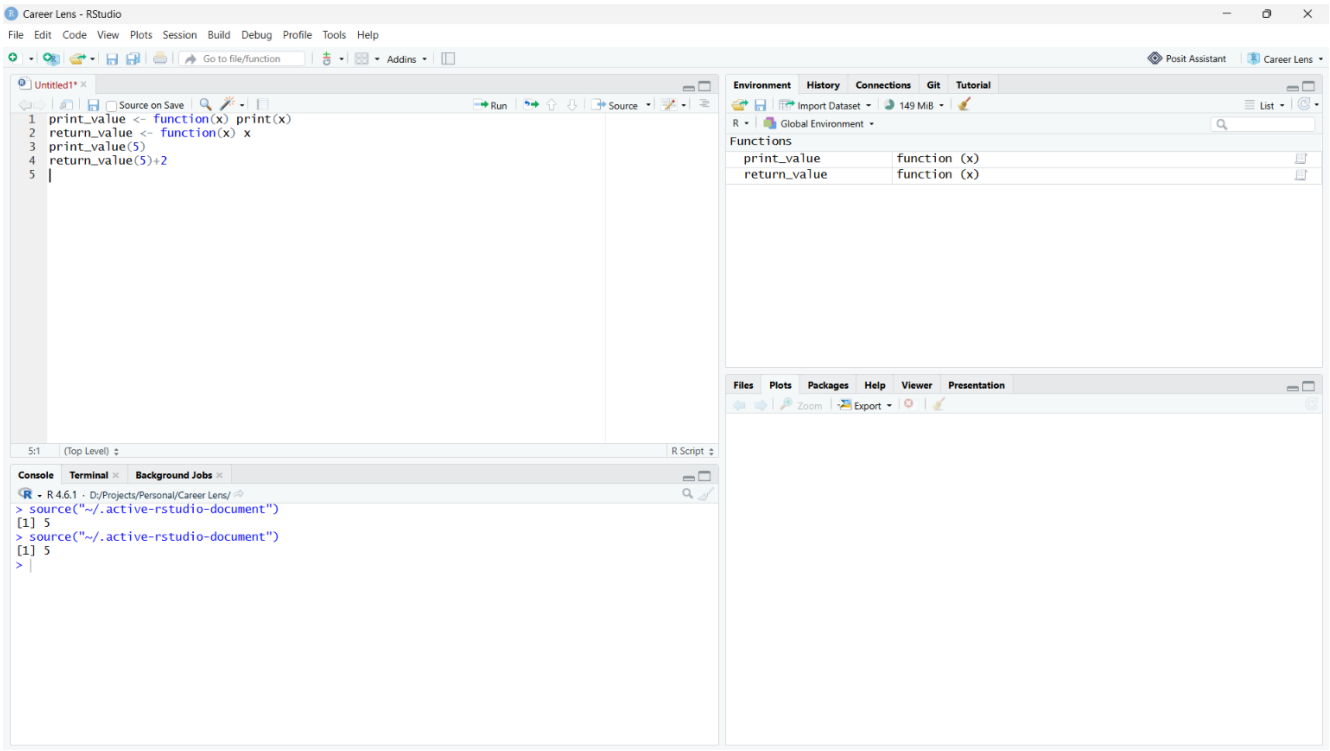
R Code:

```
print_value <- function(x) print(x)
return_value <- function(x) x
print_value(5)
return_value(5)+2
```

Expected Output:

```
[1] 5\n[1] 7
```

Result: A printed value is output only; a returned value can be used in expressions.



Q145

Task: Use sapply() and return() to label numbers low, mid or high.

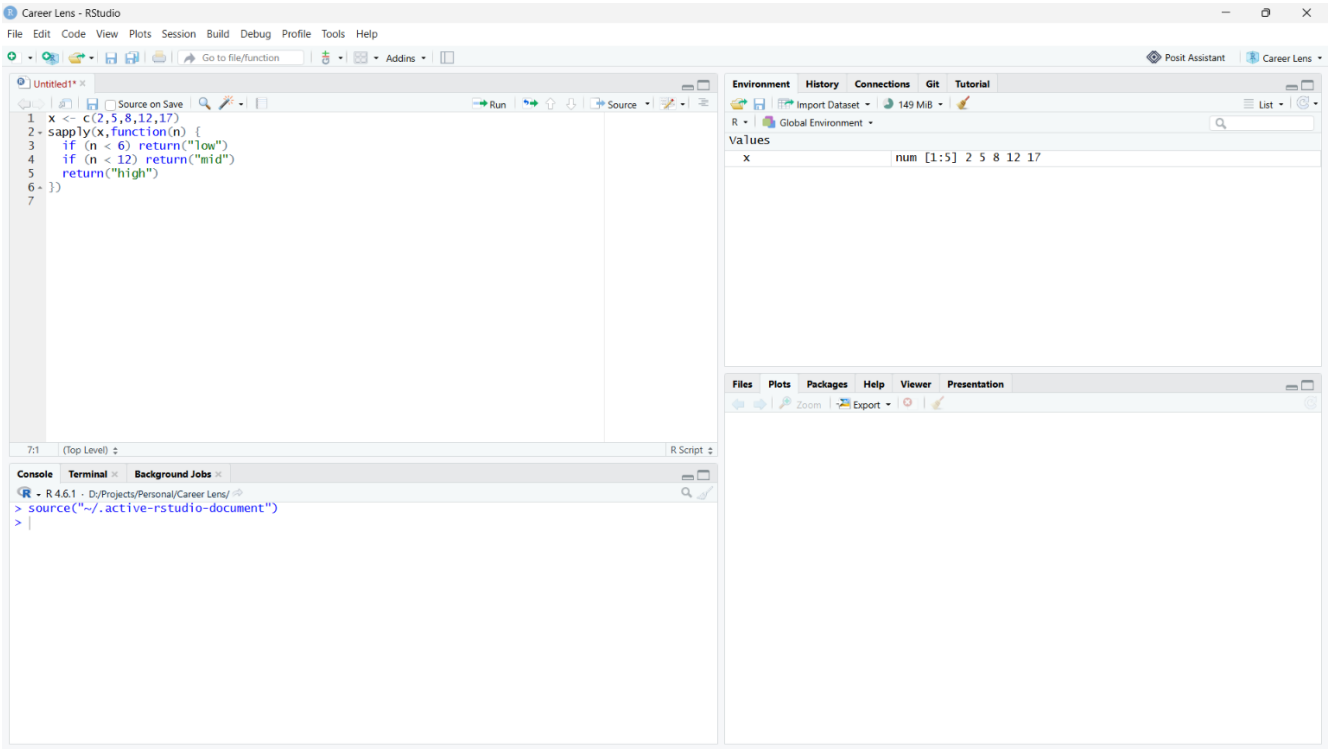
R Code:

```
x <- c(2,5,8,12,17)
sapply(x,function(n) {
  if (n < 6) return("low")
  if (n < 12) return("mid")
  return("high")
})
```

Expected Output:

```
2       5       8      12      17\n"low" "low" "mid" "high" "high"
```

Result: sapply() applies the function to every element and returns the labels.



Q146

Task: Show a function ending in a for loop returns NULL, then fix it to return the total.

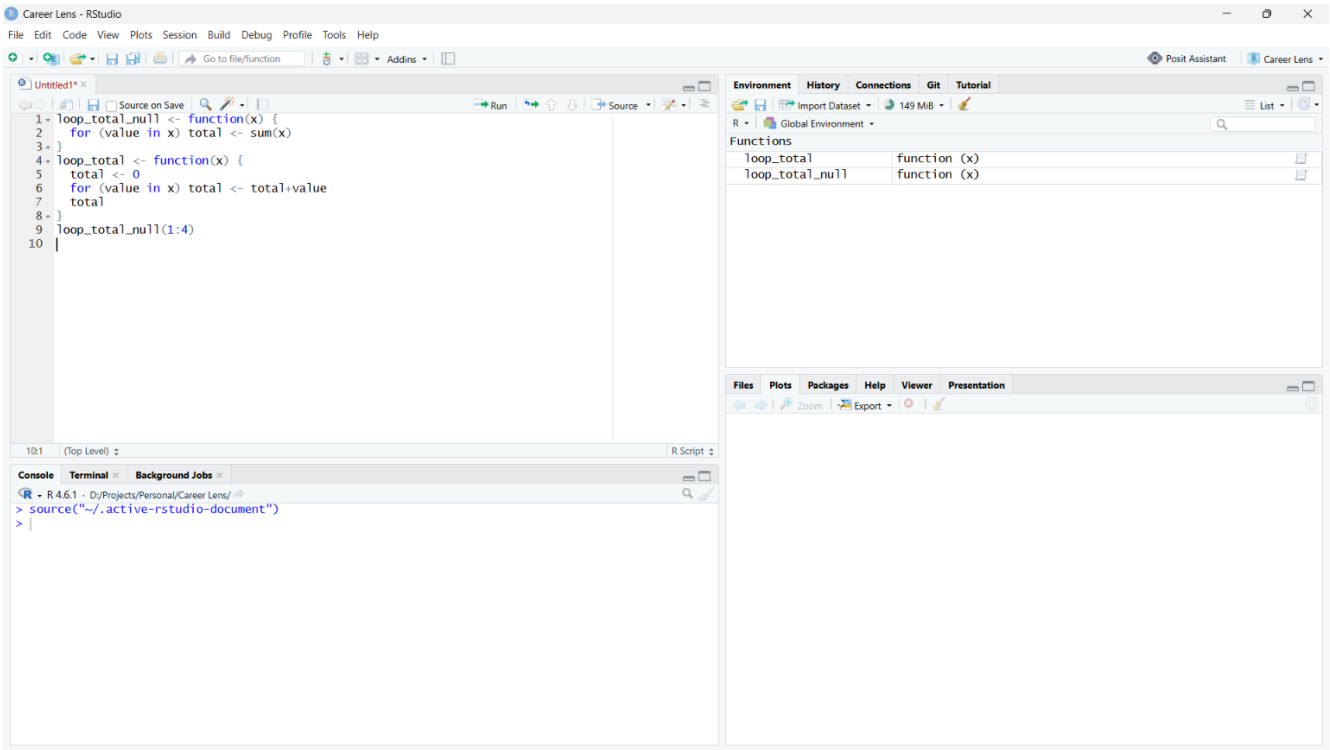
R Code:

```
loop_total_null <- function(x) {
  for (value in x) total <- sum(x)
}
loop_total <- function(x) {
  total <- 0
  for (value in x) total <- total+value
  total
}
loop_total_null(1:4)
loop_total(1:4)
```

Expected Output:

```
NULL\n[1] 10
```

Result: The loop itself returns NULL; return the accumulated total after the loop.



Q147

Task: Use grepl() with a basic regular expression, vectorised over x.

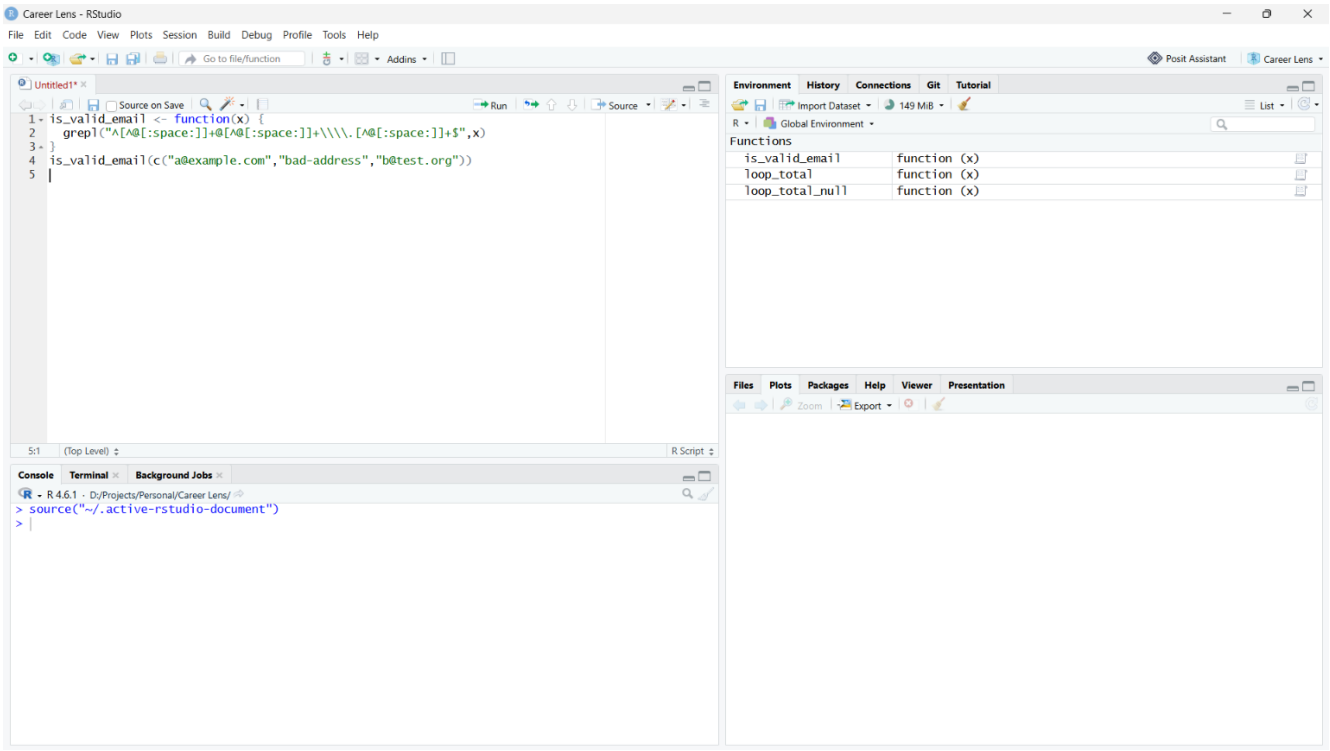
R Code:

```
is_valid_email <- function(x) {
  grepl("^[^@[:space:]]+@[^@[:space:]]+\\\\.[^@[:space:]]+$",x)
}
is_valid_email(c("a@example.com","bad-address","b@test.org"))
```

Expected Output:

```
[1]  TRUE FALSE  TRUE
```

Result: This is a basic format check, not complete email validation.



Q148

Task: Close a text connection with on.exit() and return the line count.

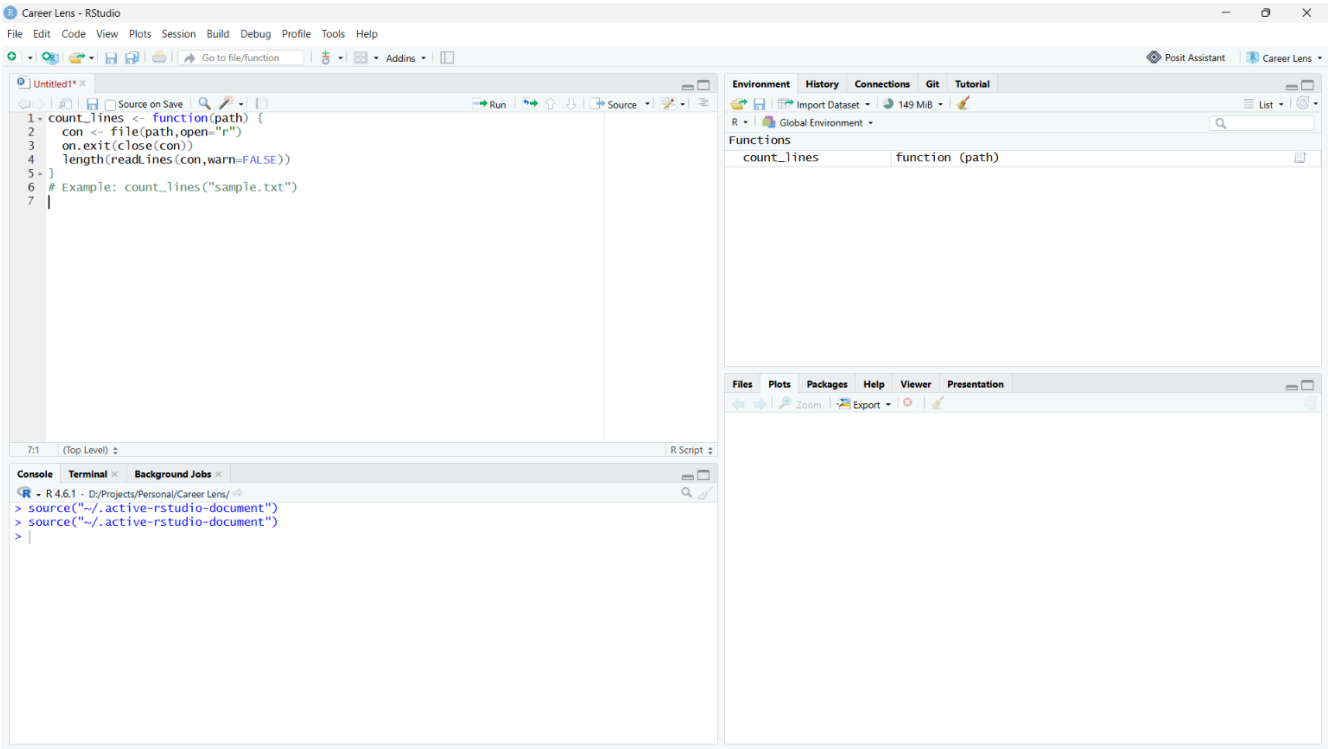
R Code:

```
count_lines <- function(path) {
  con <- file(path,open="r")
  on.exit(close(con))
  length(readLines(con,warn=FALSE))
}
# Example: count_lines("sample.txt")
```

Expected Output:

```
Returns the number of lines in the specified file.
```

Result: on.exit() ensures the connection is closed when the function exits.



Q149

Task: Show unmatched switch() returning NULL, then add a default.

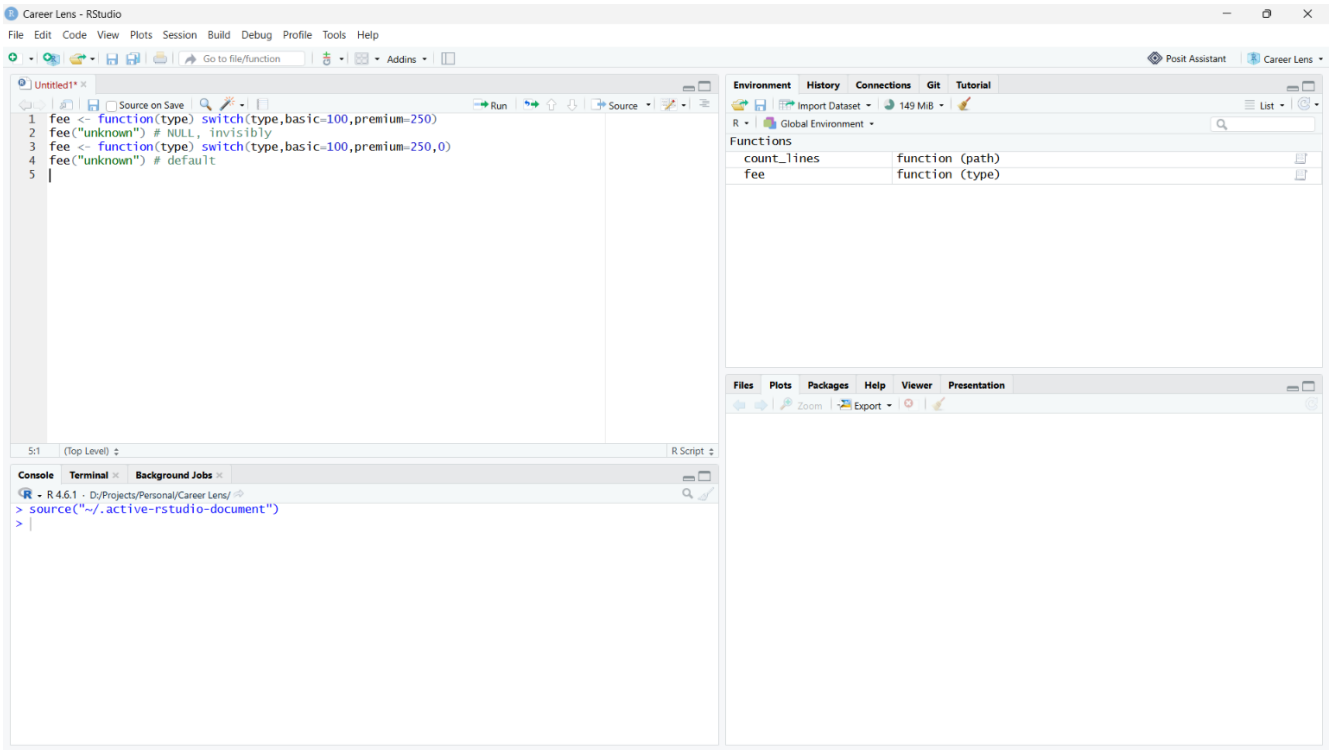
R Code:

```
fee <- function(type) switch(type,basic=100,premium=250)
fee("unknown") # NULL, invisibly
fee <- function(type) switch(type,basic=100,premium=250,0)
fee("unknown") # default
```

Expected Output:

```
First call: NULL (invisible)\nSecond call: [1] 0
```

Result: The final unnamed switch() argument is the default.



Q150

Task: Convert text to numbers while suppressing warnings for invalid text.

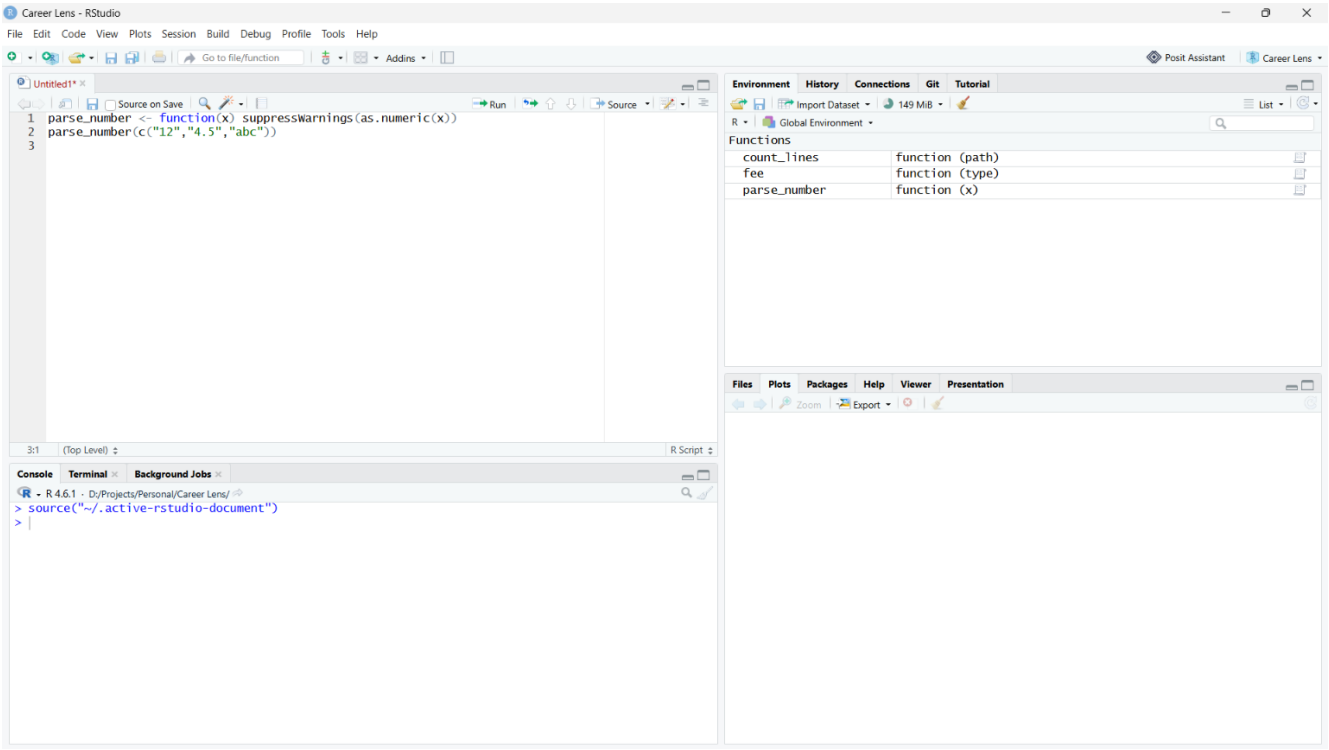
R Code:

```
parse_number <- function(x) suppressWarnings(as.numeric(x))
parse_number(c("12","4.5","abc"))
```

Expected Output:

```
[1] 12.0  4.5   NA
```

Result: Non-numeric text becomes NA and the conversion warning is suppressed.



S